DATABASE MANAGEMENT SYSTEMS | MICRO PROJECT

Library Management System

Python (Tkinter & Web REST) · SQLite 3 / MySQL 8.0 · 3NF Relational Architecture

Presented by:

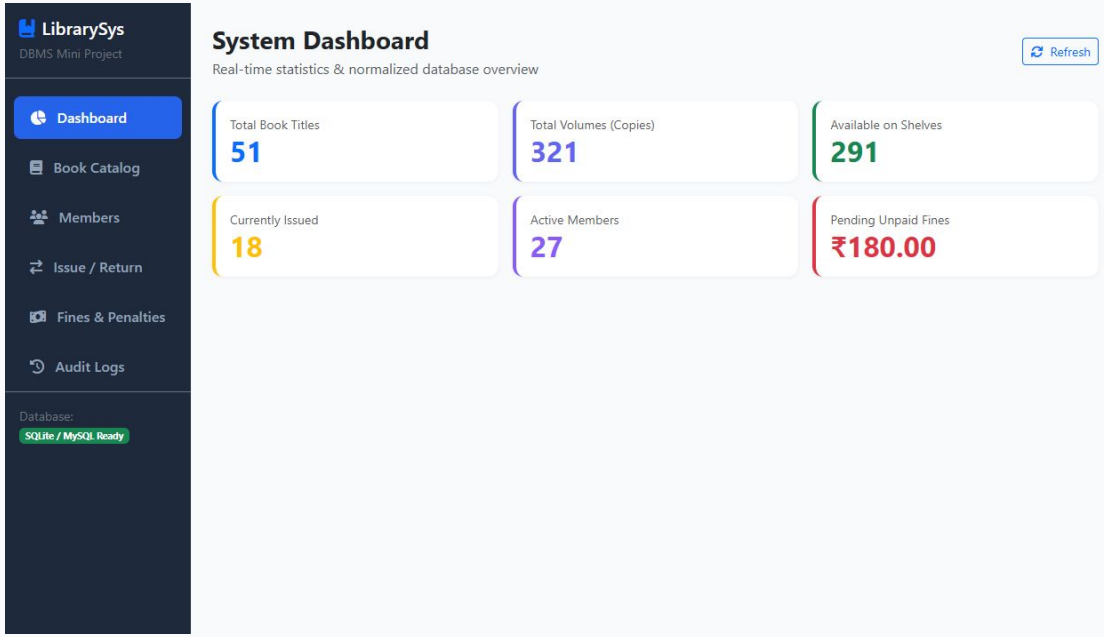
Atharva Doke (11388) · Aditya Chitimilla (11385) · Alroy Fernandes (11393)

Class: S.E. CSE | Academic Year 2026–27 | Guide: Prof. Afreen Sayed
Fr. Conceicao Rodrigues College of Engineering, Bandra (W), Mumbai

Project Overview

Automating Library Operations with Relational Integrity

- A comprehensive database application engineered to eliminate manual registers and spreadsheet errors.
- Centralizes cataloging across titles, genres, and multi-author attributions.
- Enforces patron borrowing limits (Student: 3, Faculty: 8) with real-time checkout validation.
- Automates stock decrementation on issue and restoration on return via database triggers.
- Computes late fines automatically at ₹5.00/day using date-difference trigger mechanics.
- Features dual interfaces: Desktop GUI (Tkinter) and Responsive Web App (HTML5/Bootstrap 5).



Problem Statement & Proposed Solution

Traditional Library Record Challenges

- Data Redundancy: Titles and author names duplicated across multiple checkout sheets.
- Update Anomalies: Editing author or category names leads to widespread discrepancies.
- Deletion Anomalies: Deleting past checkout records risks erasing book or member profiles.
- Stock Inconsistencies: No database-level constraints allow negative copy counts.
- Manual Overdue Calculations: Human fine calculation is prone to bias and revenue leakage.
- No Audit Trail: Lack of tamper-proof logging for catalog changes and fine receipts.

Proposed Relational DBMS Solution

- Centralized Relational Store: 3NF normalized schema eliminating redundancy across 8 tables.
- Domain Constraints: CHECK constraints ensure available_copies <= total_copies and >= 0.
- Automated Event Triggers: Database triggers calculate fines and sync stock instantly.
- Dynamic Author Creation: Register new authors on-the-fly during new book addition.
- Referential Integrity: Cascading deletes prevent orphaned child records.
- Transparent Auditing: Automatic event logging records every system modification.

Project Objectives

1. 3NF Normalized Relational Design

Eliminate insertion, update, and deletion anomalies across 8 dedicated tables with composite junction entities.

2. Active Database Triggers

Deploy autonomous triggers for automatic stock decrementation, inventory restoration, and ₹5/day fine computation.

3. Strict Referential & Domain Integrity

Implement PKs, FKs, CHECK constraints, and UNIQUE rules ensuring physical volumes never fall below zero.

4. Seamless Multi-Author Attribution

Handle Many-to-Many book-author relationships with on-the-fly author creation from the Add Book modal.

5. Dual User Interface Deployment

Provide accessible frontends (Desktop Tkinter GUI & Web Bootstrap REST Client) with real-time analytics.

Scope of the Project

Book Catalog & Authors

- Full CRUD on books & genres
- Dynamic author registration
- ISBN uniqueness validation
- Real-time shelf status

Patron Membership

- Student tier (3 books quota)
- Faculty tier (8 books quota)
- Email & phone uniqueness
- Active loan quota tracking

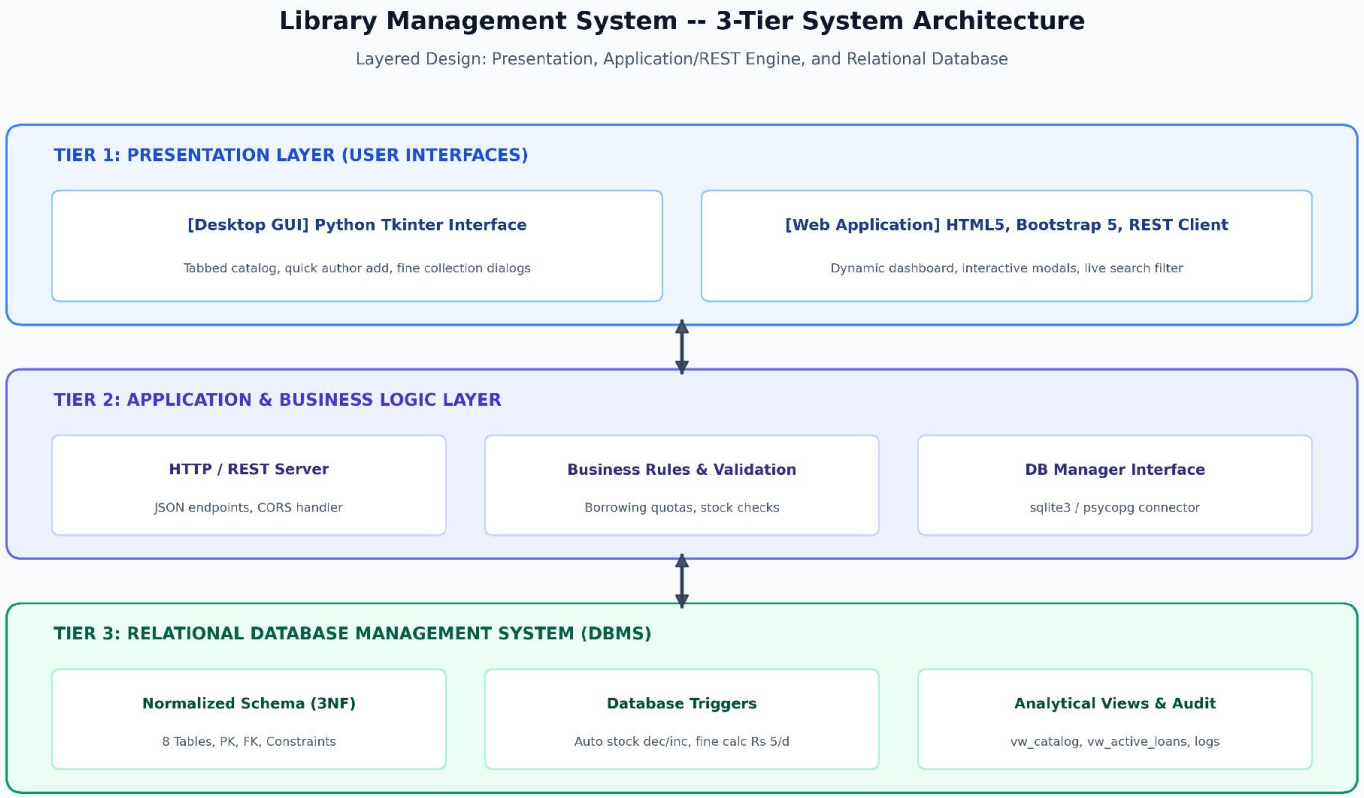
Circulation & Fines

- Issue with availability check
- Return date auto-stamping
- Trigger-based fine calculation
- Fine settlement tracking

Audit & Outside Scope

- Tamper-evident audit trail
- Pre-compiled analytical views
- Outside: Payment gateways
- Outside: RFID / Courier

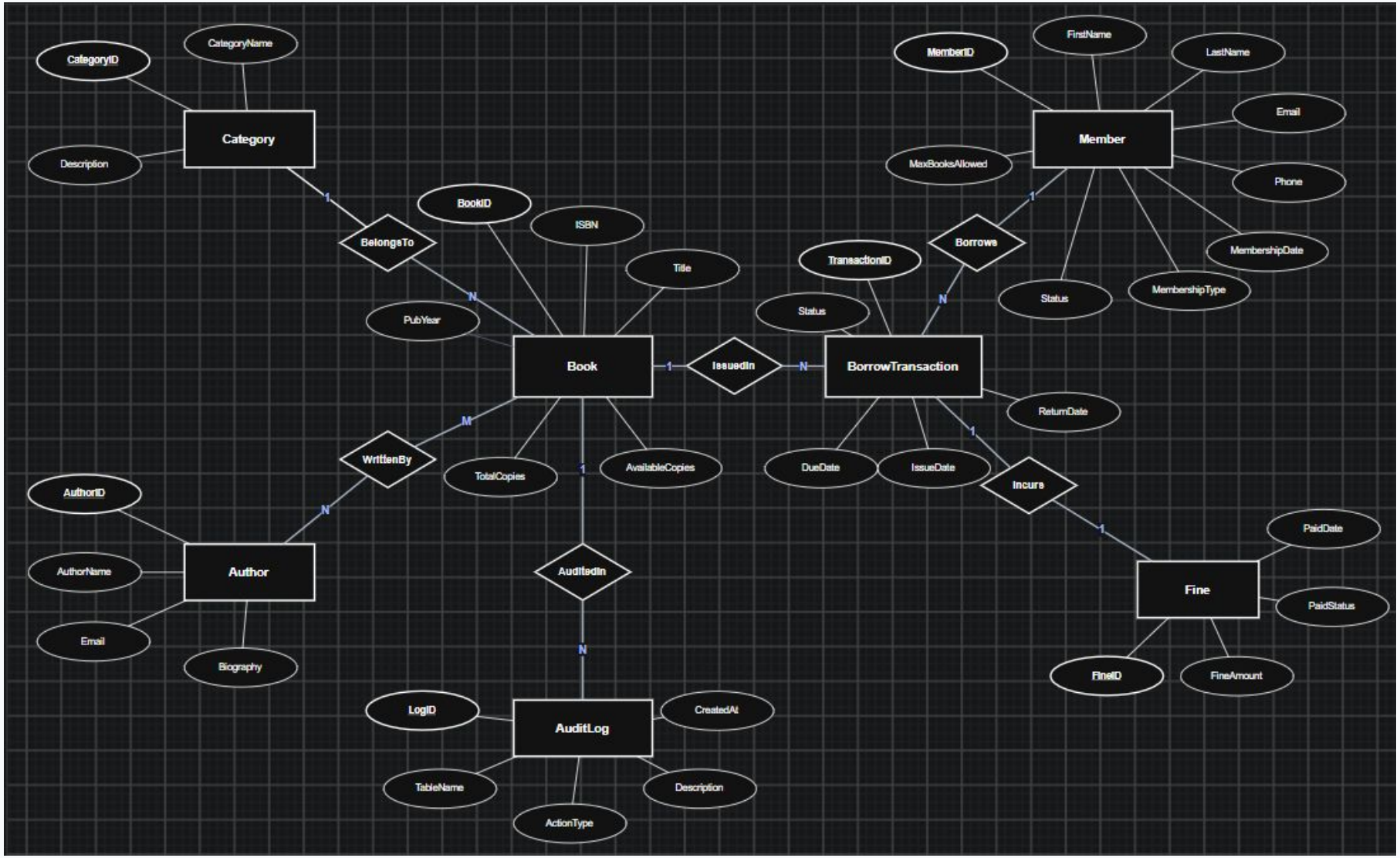
Proposed System Architecture (3-Tier)



Architectural Highlights

- Tier 1: Dual Frontends (Python Tkinter & Web Bootstrap 5).
- Tier 2: Application Controller enforcing quota validation and parameter safety.
- Tier 3: Relational DBMS Engine (SQLite 3 / MySQL 8.0).
- ACID Compliance ensures atomic borrow and return transactions.
- Pre-compiled database views optimize analytical queries.

Database Design: Entity-Relationship Model



Core Entities & Mappings

- Categories 1 : N Books (Genre classification).
- Books N : M Authors resolved via Book_Authors junction entity.
- Members 1 : N Borrow_Transactions (Quota-governed loans).
- Books 1 : N Borrow_Transactions.
- Borrow_Transactions 1 : 0..1 Fines (Optional late fee penalty).
- Audit_Logs tracks all system mutations.

Relational Schema & Key Constraints

Table Name	Primary Key (PK)	Foreign Key(s) (FK)	Key Constraints & Domain Rules
categories	category_id	None	category_name UNIQUE, NOT NULL
authors	author_id	None	author_name NOT NULL
books	book_id	category_id -> categories	isbn UNIQUE, CHECK(available <= total)
book_authors	(book_id, author_id)	book_id, author_id	Composite PK, ON DELETE CASCADE (M:N)
members	member_id	None	email UNIQUE, CHECK(max_books > 0)
borrow_transactions	transaction_id	book_id, member_id	ON DELETE CASCADE, status DEFAULT 'Issued'
fines	fine_id	transaction_id, member_id	transaction_id UNIQUE, fine_amount >= 0
audit_logs	log_id	None	Timestamped system audit trail

SQL Concepts & Implementation Matrix

DDL & Integrity

- CREATE TABLE, ALTER, DROP
- PRIMARY & FOREIGN KEY
- CHECK (copies >= 0)
- UNIQUE (isbn, email)

DML & Transactions

- INSERT INTO book_authors
- UPDATE return_date
- DELETE with CASCADE
- BEGIN, COMMIT, ROLLBACK

DQL & Joins

- INNER JOIN & LEFT JOIN
- GROUP_CONCAT authors
- WHERE, LIKE, BETWEEN
- ORDER BY & LIMIT

Triggers & Views

- Stock dec on issue
- Stock inc on return
- Overdue fine ₹5/day
- Pre-compiled analytics

SQL Queries & Database Execution Results

Query 1: Catalog with Grouped Authors

- SQL: SELECT b.book_id, b.isbn, b.title, c.category_name, GROUP_CONCAT(a.author_name) AS authors FROM books b LEFT JOIN book_authors ba ON b.book_id=ba.book_id LEFT JOIN authors a ON ba.author_id=a.author_id GROUP BY b.book_id;
- Demonstrates: Many-to-Many junction join, string aggregation, and categorization.
- Output: Cleanly lists titles with multi-author strings (e.g. 'Kernighan, Ritchie').

Query 2: Active Loans with Overdue Counter

- SQL: SELECT t.transaction_id, b.title, (m.first_name || ' ' || m.last_name) AS member_name, t.due_date, CASE WHEN DATE('now') > t.due_date THEN CAST(JULIANDAY(DATE('now')) - JULIANDAY(t.due_date) AS INT) ELSE 0 END AS days_overdue FROM borrow_transactions t JOIN books b ON t.book_id = b.book_id JOIN members m ON t.member_id = m.member_id WHERE t.return_date IS NULL;
- Demonstrates: Conditional CASE logic, Julian date arithmetic, multi-table inner joins.
- Output: Instantly identifies overdue borrowers (e.g. 33 days overdue).

Advanced SQL Queries & Aggregate Analytics

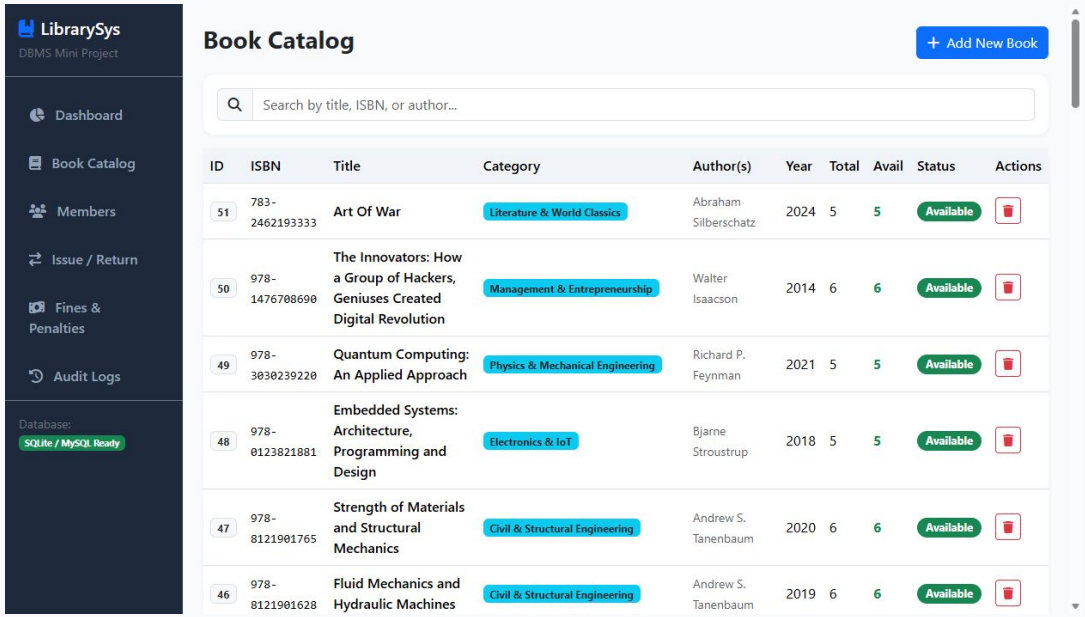
Query 3: Category Inventory Distribution

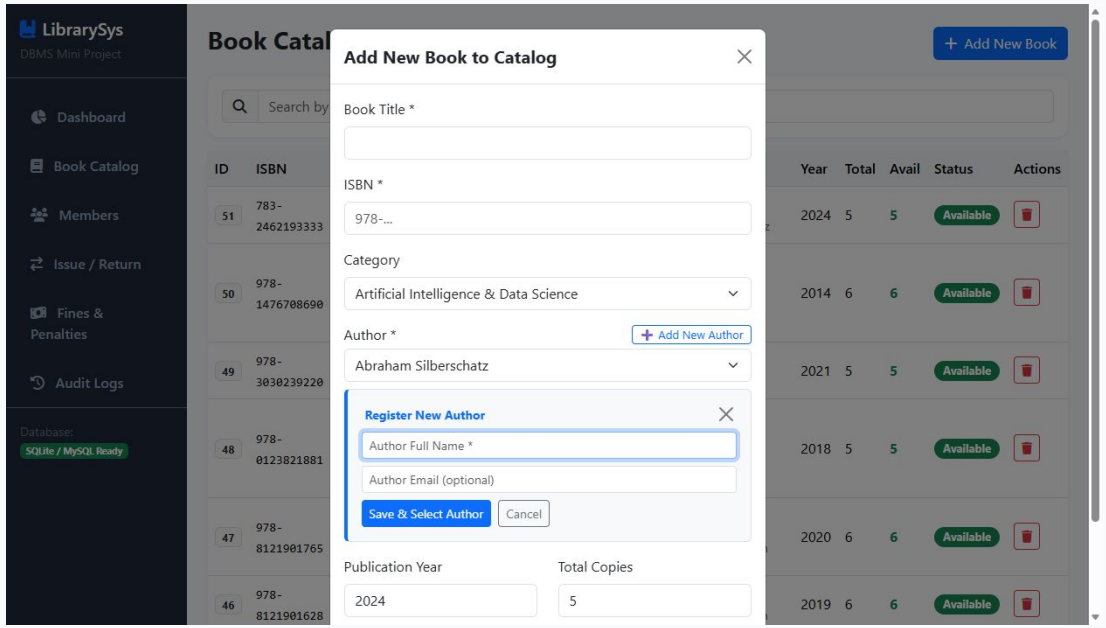
- SQL: SELECT c.category_name, COUNT(b.book_id) AS titles, SUM(b.total_copies) AS volumes, SUM(b.available_copies) AS shelf FROM categories c JOIN books b ON c.category_id = b.category_id GROUP BY c.category_id HAVING SUM(b.total_copies) >= 5 ORDER BY volumes DESC;
- Demonstrates: Multiple aggregate functions (COUNT, SUM), GROUP BY, and HAVING filtering.
- Output: CS: 26 volumes (24 shelf); SE: 14 volumes (13 shelf); Math: 7 volumes.

Query 4: Outstanding Unpaid Fines

- SQL: SELECT f.fine_id, (m.first_name || ' ' || m.last_name) AS member, m.phone, b.title, f.fine_amount FROM fines f JOIN members m ON f.member_id = m.member_id JOIN borrow_transactions t ON f.transaction_id = t.transaction_id JOIN books b ON t.book_id = b.book_id WHERE f.paid_status = 'Unpaid';
- Demonstrates: Four-table JOIN, currency formatting, status filtering.
- Output: Accurately produces recovery list for administrative follow-up.

Application Interface & Dynamic Author Creation

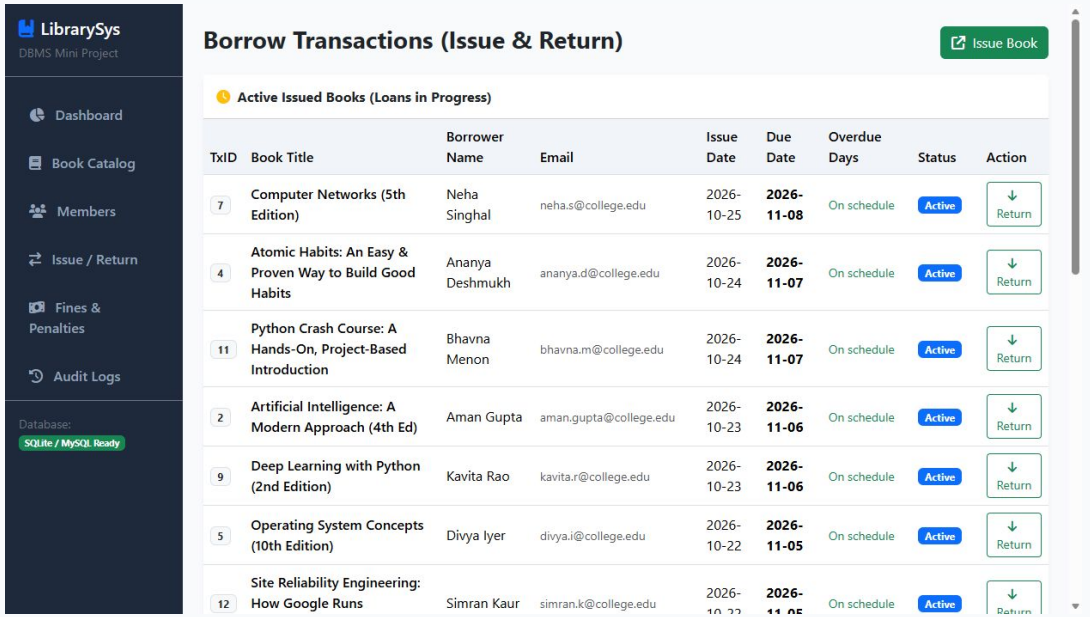


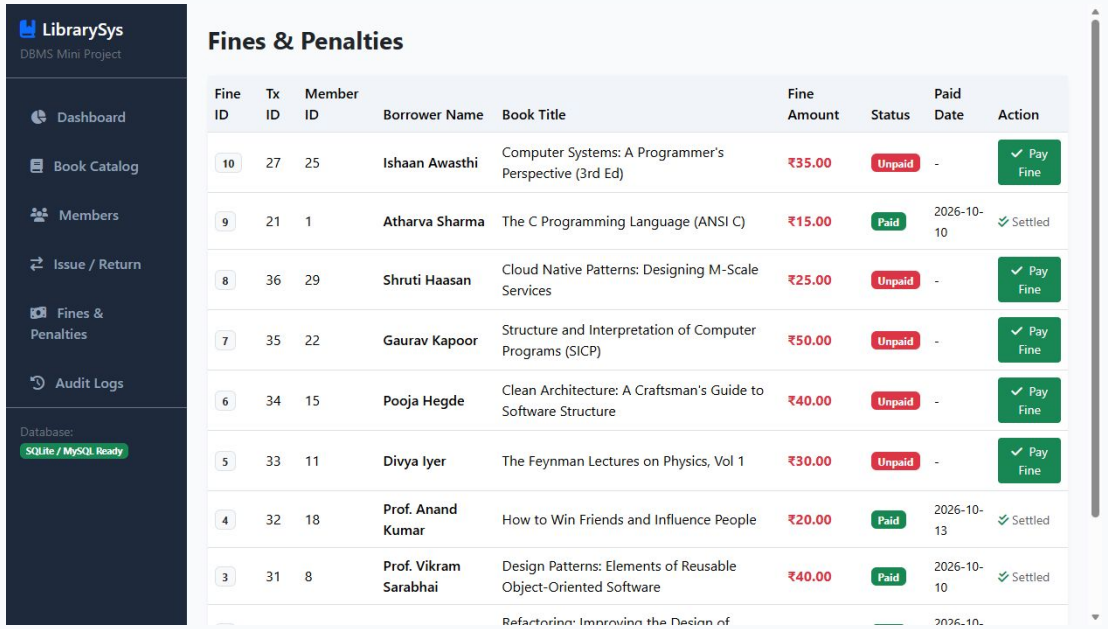


Key Feature: On-The-Fly Author Registration in Add Book Dialog

- Librarians can click '➕ Add New Author' directly inside the book entry dialog.
- Creates author record, registers audit trail, and auto-selects newly created author seamlessly.

Circulation Desk, Fines & Audit Reports





Real-Time Circulation & Fine Settlement Workflows

- Instant one-click book returns with automated trigger-driven stock and fine computation.
- Transparent fine collection tracking and tamper-evident audit logging for all database events.

Expected Outcome & Conclusion

Achieved Project Outcomes

- Centralized & Persistent Data: Fully normalized 3NF relational schema.
- Eliminated Data Redundancy: Relational separation across books, authors, members, and transactions.
- Automated Business Logic: Event triggers handle inventory counts and fine assessments without human error.
- Zero Inventory Corruption: Database constraints prevent negative book balances.
- Tamper-Proof Audit Logging: Every catalog change and payment is logged with exact timestamps.

Conclusion & DBMS Concepts Demonstrated

- Combines Python, SQLite, and modern Web technologies to solve real-world library administration challenges.
- Demonstrated core DBMS principles: Normalization (1NF to 3NF), Referential Integrity, DDL/DML/DQL mastery.
- Proved event-driven automation via SQLite Triggers and Views.
- Delivered dual intuitive interfaces ensuring reliable, multi-client access.